

Introduction

LLM and agents have taken quite a lot of space in the last couple of years. They have proven to be astonishing in predicting the most probable outcome to an input. The input can be a question that a user wants the answer to: think about what do I want to do for dinner, what should I do as a training program, etc... It can also be a problem the user has, can you help me solve this Excel formula, can you explain it in depth, etc... And lately, we can build tools and agents (maybe even agents to communicate with one another in a near future) to help us accomplish task at a speed never seen. Well what if we could build a pipeline of artificial intelligence deployable units (AIDU) that can be given a general task (this can consist of any *software* task) and split up to work amongst n number of sub-agents to start building or solving said task.

I do make the distinction of building and solving above, because this is only from a human readable perspective. A machine, or in this case an AI agent, doesn't care if it's building nor if it's solving something. It currently only knows of some inputs (step 1), crunching some code - calling apis - calling tools (step 2 ... $n - 1$) and the outputs (step n). The inputs and the outputs are what we humans care about. In the case of AI Agents, the tools and the way it got to the end goal of giving us the outputs is not that important (later notes about doing it ethically), we care that it reached the end goal, or not. If it didn't reach the end goal, the AI Agent should be able to provide a detailed explanation of why it was not able. Maybe it needed human input, or maybe it was something else (limited compute power).

Whenever we encounter an issue and we want to troubleshoot it, it can be digital or physical, we should always refer to the below guide.

Troubleshooting - This has been taken from the CompTIA core 1 course

Identify the problem

What is the problem? What are the issues you are encountering? You have provided some inputs, they can be physical or digital, and the outputs are not what you are expecting.

Establish a theory

Establish a theory of what we can do to fix the problem. Make a list of all possible causes (start

with easy and straightforward ideas, to more complex ones). Don't forget *Occam's razor* (The simplest explanation is often the most likely). Check internal resources, known issues (did we already encounter this kind of issue in the past?).

Test the theory

Test out the theory and determine the next steps to resolve the problem. If the theory didn't work, circle back to the beginning (did we correctly understand what the issue is? did we miss anything). Re-establish a new theory and retry. Do this a couple of times (let's say three times). If you can't find resolve the issue, escalate. Call an expert.

Create a plan of action

Build the plan to solve the issue. In the case of software, this could be to apply a patch, adding a feature flag (this can be a temporary fix, or maybe a permanent one...) or adding a new component.

Implement the Plan

Fix the issue, implement the change control needed. In the case of software, this means doing the actual coding of what fixes the issue.

Verify full system functionality

A good piece of code is as good as it's tests. Tests should never be neglected. If have taken an input that wasn't correct, implemented a change that fixes the issue. Write out a test to correct test again the issue. The test is always part of the plan.

Document Findings

Document what the issue was, what we did to implement a fix and what tests we ran (maybe slap in a test report). This will now be part of the knowledge base of the next person (or agent) to use in their next analysis of troubleshooting an issue.

This next part is about the organizing any project or idea we what to tackle. Think of tackling a project you want to do. Let's say a tic-tac-toe game. The below guide should give insight in what needs to be done to reach our goal. I have been highly influenced in the triple AAA method after encountering Panam in a video game called Cyberpunk 2077. The references have been added accordingly.

Panam's triple A rule (from Cyberpunk 2077)

[Reference](#)

Assessment

This first step involves gathering information, understanding the problem at hand, and knowing

what resources you have available. Assessing the situation first allows you to gain a clear understanding of what you're facing, reducing the chance of encountering unexpected problems later on.

Assembly

After understanding the situation, you gather the resources needed to tackle the problem. This could involve acquiring new tools, learning new skills, or even assembling a team of people to assist you. This step is about preparation and ensuring you're ready to take on the challenge.

Action

Once you've assessed the situation and assembled your resources, it's time to act. You execute your plan to solve the problem or achieve your goal. This principle emphasizes the importance of taking decisive action based on thoughtful preparation.

Bonus - Improvisation

This encourages flexibility and adaptability, important skills in any aspect of life. It's a reminder that even the best-laid plans can encounter obstacles and that we must be prepared to adjust and adapt when needed.

Now how do we combine the troubleshooting and AAA guides to build out our orchestration AIDU pipeline?

Let's start with a concret example. Let's say we want to build simple tic-tac-toe web page that has a simple frontend and a simple backend. We, as humans, should start by defining all the components we want to build. For the sake of brevity, I will use a local folder containing files that are considered as tasks. Let's image this sort of structure:

```
TTT-frontend/  
  TTT-home-page.md  
  TTT-home-page-tests.md  
  TTT-components.md  
  TTT-components-tests.md  
  [...] # any other meaningful task you may think of  
  # optional there can be an agent.md file which gives out additional  
instructions and information to the agent  
TTT-backend/  
  TTT-core-game-logic.md  
  TTT-core-game-logic-tests.md  
  TTT-rest-api.md  
  TTT-rest-api-tests.md  
  [...] # any other meaningful task you may think of
```

```
# optional there can be an agent.md file which gives out additional
instructions and information to the agent
```

I use TTT as reference for tic-tac-toe, but this can be any three letter word you might find across Jira, Linear, Mira, or any similar product management platform.

Note: I have not went deep in this case by adding a database component, persistence of user sessions, authentication validations, etc. I know of numerous ways to improve this game.

Now let's imagine that we have command line that starts up the AIDU, we will go in detail in a bit.

```
aidu deploy claude TTT
```

Here we are using `claude` as an assistant but realistically, it can any kind of agents, they can be pluggable and modular. We then pass in `TTT` which is the tic-tac-toe game we want to build.

Now when the command line fires, we will start by deploying an AAA manager.

AAA Manager AI Agent

The AAA manager will be given the general instructions of *Panam's triple A rule* with any information on what we plan on building.

This step is really important that it's well written out. The instructions and informations we give the AAA manager need to be rich but succinct at the same time. It should consist of a global idea of what the project should be doing this and that. It should not be doing this nor should it be doing that.

The AAA manager knows that it can use any sub-agent at it's disposition to build out the instructions we have given it.

The AAA manager will then start by applying the AAA guide.

1. It will start by reading all the files under `TTT-frontend` and `TTT-backend` .
 - Again, all the tasks in `TTT-frontend` and `TTT-backend` should be rich of meaningful information. No need to be explicit on *how* to build out a task. The sub-agents will be smart enough to know of the best paths and decisions to take to build out the best outcome for any task.
2. It will assemble the team it needs to build out the tic-tac-toe game. In this case, it will act like a product manager that needs to *spawn* developers to start working on various tasks. In the AIDU case, it will start to *spawn* various sub-agents and assign them tasks to do.
3. The AAA manager will then give a GO to all the sub-agents to start working on their individual tasks. A good future improvement would be to have the sub-agents communicate

to one another. Like pair programming on various task. Like us humans, when we pair program with pairs, it may spark a good idea on building something differently.

Once the sub-agents have completed tasks and the tests are running as expected, the AAA manager is tasked to make sure that the task the sub-agents have accomplished work as expected. They will be encouraged to start up QA-agents to run test and then compile any document findings the latter agents report.

Sub-agents

The sub-agents will be given the general instructions of troubleshooting with any information what what task they have to accomplish. Ideally they would also be given a very succinct description of what the overall goal is, i.e. what is the end goal of the small unit they are building.

Sub-agents will be encouraged to always test out any part code they are building. And if they get stuck on something, like not able to do something, they can first escalate to the AAA manager about the issue they can't seem to fix.

The AAA manager can make multiple decisions at this point:

- *spawn* a new agent with a *cleaner* context window and try to take the same issue again
- *spawn* a sort of reviewer agent that can start by checking the work done and maybe propose a new idea
- notify a human that it needs help

The AAA manager will be smart enough to know how many sub-agents it needs to build out the project.

QA-agents (They can also be named tester-agents)

QA-agents have precisely two tasks:

1. Test out any feature or task they have been given
2. Document the findings

Once the AAA Manager, has enough assurance that the project has been build. It can load out a binary that a human can test and wait for the human to provide additional input. The loop then goes over and over again until the human has the desired outputs.

Ethical and security considerations

I have mentioned that *the tools and the way it got to the end goal of giving us the outputs is not that important*. To be more precise: any agent should never use any illegal or evil way to solve an issue. This would mean:

- No cheating out tests to prove that everything worked as expected when it did not
- Respect any of the OWASP Top Ten security considerations (<https://owasp.org/www-project-top-ten/>). This means to never provide with a solution to a task which could be vulnerable to any security bug
- No bypassing any laws or doing anything illegal